AIMERZ.ai
Aim.Act.Achieve

NOTES

Collections Framework/API - IV

Working with Utility Class (Arrays & Collections)

Utility Classes in Java

Utility classes in Java make our life easier by providing ready-made methods to perform common tasks, like sorting, searching, or working with arrays and collections. These classes save time and effort by offering static methods that simplify our code.

The most commonly used utility classes in Java are:

- ➔ **Collections Utility Class**
- ➔ **Arrays Utility Class**
- ➔ **Objects Utility Class**

Let's dive into each class with explanations, real-world analogies, examples, and practical use cases.

Collections Utility Class

The Collections class belongs to the **java.util** package and provides various static methods to work with **Collections** like List, Set, and Queue. It is a one-stop helper for operations like sorting, reversing, shuffling, finding max/min, and more.

Key Methods and Their Uses

Method	Description
sort(List<T> list)	Sorts the given List in ascending order.
sort(List<T> list, Comparator<? super T> c)	Sorts the given List based on a custom comparator.
reverse(List<?> list)	Reverses the order of elements in the given List.
shuffle(List<?> list)	Randomly shuffles the elements of the given List.

min(Collection<? extends T> coll)	Returns the minimum element from the given Collection.
max(Collection<? extends T> coll)	Returns the maximum element from the given Collection.
binarySearch(List<? extends T> list, T key)	Performs a binary search to find the specified key in a sorted List.
binarySearch(List<? extends T> list, T key)	Performs a binary search to find the specified key in a sorted List.
copy(List<? super T> dest, List<? extends T> src)	Copies elements from the source List to the destination List.
swap(List<?> list, int i, int j)	Swaps elements at specified indices i and j in the given List.
frequency(Collection<?> c, Object o)	Returns the frequency of the specified element o in the given Collection.
unmodifiableList(List<? extends T> list)	Returns an unmodifiable view of the given List (cannot add/remove elements).

Code Example: Demonstrating All Methods

Java
```
import java.util.*;

public class CollectionsUtilityDemo {
    public static void main(String[] args) {
        // Sample List
                                List<String>    names    =    new
ArrayList<>(Arrays.asList("Alice",  "Bob",  "Charlie",  "Alice",
"Eve"));
        System.out.println("Original List: " + names);

        // 1. Sort List in ascending order
```

```
        Collections.sort(names);
        System.out.println("Sorted List: " + names);

        // 2. Sort List with Comparator (Descending Order)
        Collections.sort(names, Collections.reverseOrder());
            System.out.println("Sorted List (Descending): " +
names);

        // 3. Reverse List
        Collections.reverse(names);
        System.out.println("Reversed List: " + names);

        // 4. Shuffle List
        Collections.shuffle(names);
        System.out.println("Shuffled List: " + names);

        // 5. Find Minimum Element
        String minName = Collections.min(names);
        System.out.println("Minimum Element: " + minName);

        // 6. Find Maximum Element
        String maxName = Collections.max(names);
        System.out.println("Maximum Element: " + maxName);

        // 7. Binary Search (List must be sorted first)
        Collections.sort(names); // Sorting for binary search
                int index = Collections.binarySearch(names,
"Charlie");
        System.out.println("Index of 'Charlie': " + index);

        // 8. Copy Elements from one List to another
                        List<String> copyList = new
ArrayList<>(Arrays.asList("", "", "", "", ""));
          Collections.copy(copyList, names); // Source: names,
Destination: copyList
        System.out.println("Copied List: " + copyList);

        // 9. Swap Elements
```

```
        Collections.swap(names, 0, 4); // Swapping first and last elements
        System.out.println("List after Swap: " + names);

        // 10. Frequency of an Element
        int frequency = Collections.frequency(names, "Alice");
        System.out.println("Frequency of 'Alice': " + frequency);

        // 11. Create an Unmodifiable List
        List<String> unmodifiableList = Collections.unmodifiableList(names);
        System.out.println("Unmodifiable List: " + unmodifiableList);
        // unmodifiableList.add("NewName"); // This line would throw an UnsupportedOperationException
    }
}
```

Output

```Java
Original List: [Alice, Bob, Charlie, Alice, Eve]

Sorted List: [Alice, Alice, Bob, Charlie, Eve]

Sorted List (Descending): [Eve, Charlie, Bob, Alice, Alice]

Reversed List: [Alice, Alice, Bob, Charlie, Eve]

Shuffled List: [Eve, Bob, Alice, Charlie, Alice]

Minimum Element: Alice

Maximum Element: Eve

Index of 'Charlie': 3

Copied List: [Alice, Alice, Bob, Charlie, Eve]
```

```
List after Swap: [Eve, Alice, Bob, Charlie, Alice]

Frequency of 'Alice': 2

Unmodifiable List: [Eve, Alice, Bob, Charlie, Alice]
```

Arrays Utility Class

The Arrays class is another utility class in **java.util**. It provides static methods to manipulate arrays, such as sorting, searching, and converting arrays to lists.

Key Methods and Their Uses

Method	Description	Real-World Analogy	Example
asList(T... a)	Converts an array to a list.	Turning a menu into a checklist.	Convert an array of fruits into a list.
sort(T[] a)	Sorts an array in ascending order.	Organizing utensils in order of size.	Sort an array of numbers.
binarySearch(T[] a, T key)	Searches for an element in a sorted array.	Looking for a phone number in a phonebook.	Search for a specific number in an array.

copyOf(T[] original, int newLength)	Copies an array and changes its size.	Adding blank pages to a notebook.	Copy a student list with extra slots.

Example 1: Converting an Array to a List

```Java
import java.util.Arrays;
import java.util.List;

public class ArraysAsListExample {
    public static void main(String[] args) {
        String[] fruits = {"Apple", "Banana", "Cherry"};

        // Convert array to list
        List<String> fruitList = Arrays.asList(fruits);

        System.out.println("Fruits: " + fruitList);
    }
}
```

Analogy: Imagine turning a recipe into a checklist for easy use.

Example 2: Sorting an Array

```Java
import java.util.Arrays;

public class ArraysSortExample {
    public static void main(String[] args) {
        int[] numbers = {5, 2, 8, 3, 1};

        // Sort the array
```

```
        Arrays.sort(numbers);

 System.out.println("Sorted Array: " +
Arrays.toString(numbers));
    }
}
```

Real-World Use Case: Sorting exam marks in ascending order to find the topper.

Example 3: Copying and Extending an Array

```Java
import java.util.Arrays;

public class ArraysCopyOfExample {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3};

        // Extend array size
        int[] newNumbers = Arrays.copyOf(numbers, 5);

        System.out.println("Extended Array: " +
Arrays.toString(newNumbers));
    }
}
```

Analogy: Imagine copying a list of attendees and leaving space for future entries.

Objects Utility Class

The Objects utility class (introduced in Java 7) provides methods for working with objects, such as checking for null values, comparing objects, or generating hash codes.

Key Methods and Their Uses

Method	Description	Real-World Analogy	Example
requireNonNull(Object obj)	Throws an exception if the object is null.	Ensuring a guest list has no missing names.	Validate a username is not empty.
equals(Object a, Object b)	Compares two objects for equality.	Checking if two gifts are identical.	Compare two user profiles.
hash(Object... values)	Generates a hash code for multiple objects.	Creating a unique ID for a library book.	Generate hash for a user's data

Example: Null Check

```Java
import java.util.Objects;

public class ObjectsExample {
    public static void main(String[] args) {
        String name = null;

        // Check for null
        try {
```

```
                    Objects.requireNonNull(name, "Name cannot be
null");
        } catch (NullPointerException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Real-World Use Case: Ensuring required fields (like username or email) are filled before submitting a form.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan